

Universidade Federal de Minas Gerais (UFMG)

Engenharia de Controle e Automação

Sistema Automatizado de Inspeção de Túneis

Trabalho Final - Automação em Tempo Real

Alunos:

Ana Beatriz Soares Cardoso – 2022108170

Julia Pereira Maia Ribeiro – 2021014864

Pedro Eduardo Alvino Tapia – 2021031084

Belo Horizonte

2026

Índice

1	Introdução	1
2	Estruturação: Arquitetura e Comunicação de Dados	1
3	Teste Realizados e Resultados	1
3.1	Mapeamento de Superfície e Perfilamento do Túnel	1
3.2	Monitoramento Dinâmico de Amostragem e Overhead do Controlador PID	2
3.3	Validação de Jitter sob Intervenção Dinâmica de Modo (Override)	2
3.4	Execução Concorrente de Tarefas Periódicas e Assíncronas	3
3.5	Análise de Carga Real em Cenário Crítico de Inspeção Visual	4
3.6	Levantamento dos Piores Tempos de Execução (C_i)	6
3.7	Modelo Teórico de Escalonabilidade Avançado	6
3.8	Cálculo do Período Mínimo das Tarefas Cíclicas ($T_{\min}$)	7
3.9	Discussão Teórica e Restrições de Hardware	7
4	Conclusão e Discussões	8
5	Apêndice	8

1 Introdução

descreva o problema de forma sucinta, com uma figura ilustrativa. Não copie a mesma figura usada na especificação do trabalho. Descreva os objetivos do projeto.

2 Estruturação: Arquitetura e Comunicação de Dados

refinamento da proposta entregue na primeira etapa com descrição detalhada das ferramentas de sincronização utilizadas.

3 Teste Realizados e Resultados

Esta seção apresenta os resultados experimentais obtidos durante a execução do sistema integrado de inspeção. Os testes foram projetados para validar o comportamento dinâmico do robô, o determinismo das tarefas cíclicas, o tratamento de eventos assíncronos e a fidelidade do mapeamento sensorial.

3.1 Mapeamento de Superfície e Perfilamento do Túnel

O primeiro teste validou o funcionamento do sensor LIDAR e o envio de dados via protocolo MQTT para a interface de supervisão remota, operando em Modo Automático com velocidade nominal configurada para 1.0 m/s.

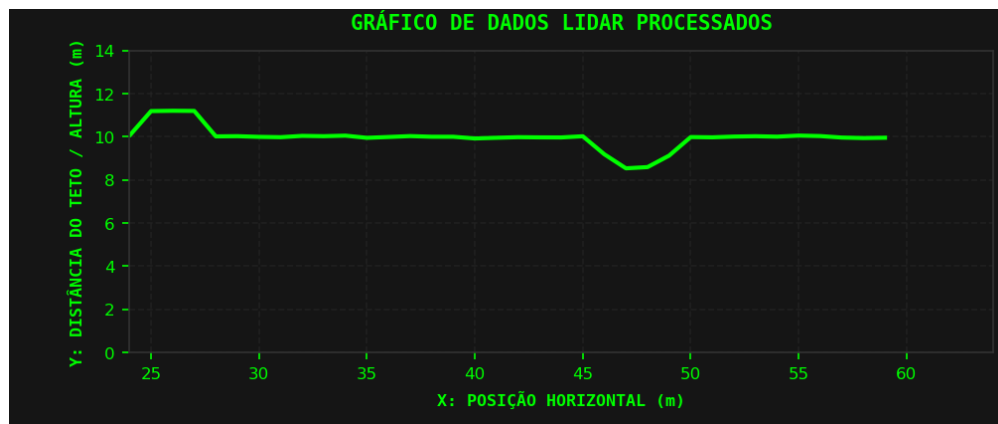


Figura 1: Grafico Lidar

Análise do Resultado: O gráfico gerado de forma autônoma pela GUI comprova o correto funcionamento do algoritmo de filtragem por média móvel. O perfil dinâmico revela um trecho reto inicial perfeitamente estabilizado na altura ideal de 10.0 m. Por volta da posição horizontal de 25 m, o filtro atenua ruídos de medição e registra com precisão o degrau de um buraco estrutural (aumento da distância lida para mais de 11.0 m). Posteriormente, o robô retorna ao regime reto estável e, na marca de 45 m, mapeia uma saliência contínua (redução da distância lida para aproximadamente 8.5 m), validando o processamento sensorial síncrono.

3.2 Monitoramento Dinâmico de Amostragem e Overhead do Controlador PID

Para avaliar a estabilidade temporal do núcleo matemático implementado em C++, coletou-se uma amostragem contínua de 1050 ciclos consecutivos de processamento da malha de controle de velocidade em regime estável.

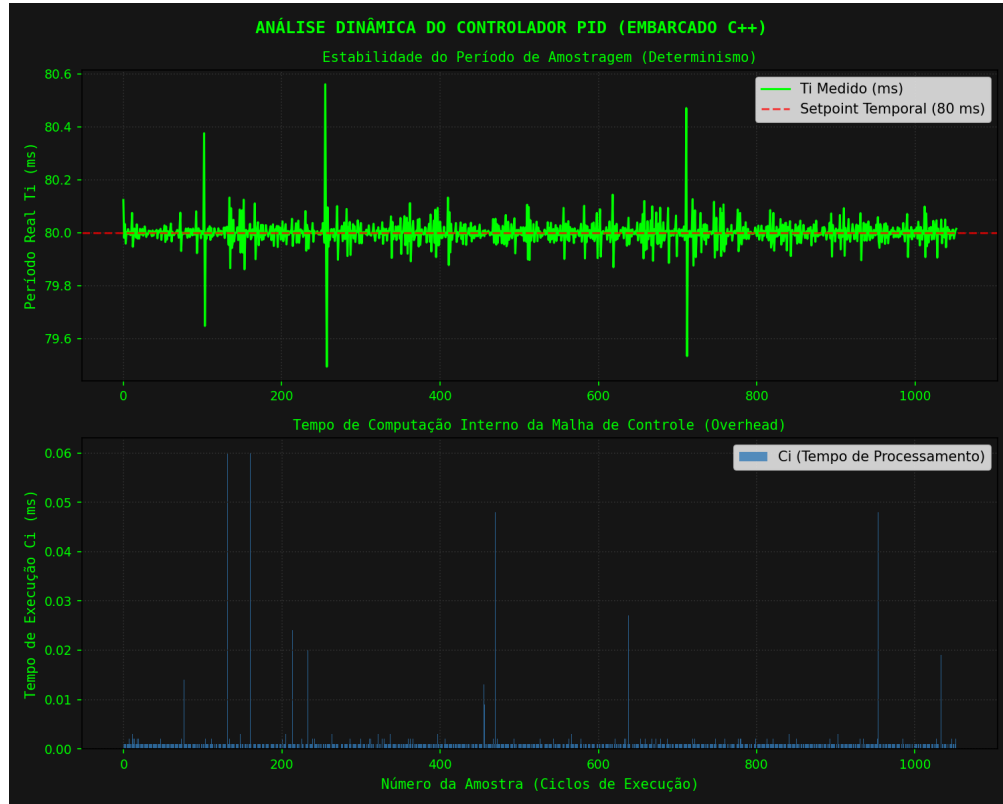


Figura 2: Tempos de cada tarefa síncrona

Análise do Resultado: O gráfico superior comprova o estrito determinismo temporal obtido no sistema. A linha verde (Período Real Medido T_i) permanece linearmente ancorada em cima do setpoint temporal nominal de 80 ms. As flutuações de clock (*jitter*) registradas ao longo de toda a simulação contiveram-se estritamente entre 79.5 ms e 80.6 ms, determinando uma margem de variação inferior a 0.75%, o que garante a estabilidade do controlador discretizado.

O gráfico inferior confirma que o tempo de computação pura (C_i) para processar as equações do PID e a trava anti-windup é desprezível, mantendo-se na faixa de 0.001 ms a 0.002 ms. Os raríssimos picos de overhead de sistema que atingiram 0.06 ms decorrem de chamadas concorrentes de I/O do console, demonstrando que a thread de controle consome menos de 0.075% de banda da CPU, deixando o processador livre para as tarefas esporádicas.

3.3 Validação de Jitter sob Intervenção Dinâmica de Modo (Override)

Este teste validou a capacidade do sistema de processar com segurança a transição instantânea de modos de operação quando o robô está em modo automático e o operador assume o controle manual

de emergência.

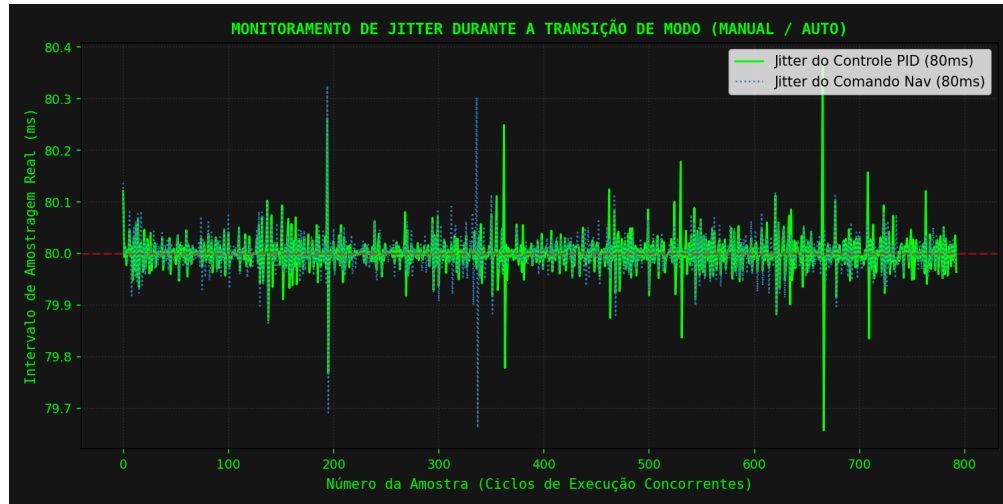


Figura 3: Transições de automatico para manual

Análise do Resultado: O gráfico demonstra o comportamento temporal concomitante das threads de Comando (**Comando Nav**) e Controle (**Controle PID**) durante o exato ciclo em que o operador intercepta o robô. A estabilidade das curvas comprova que, mesmo com a injeção assíncrona de dados via pacotes MQTT alterando as flags de direção do buffer, o período real de ambas as tarefas cíclicas manteve-se estável na marca de 80 ms. O desvio de amostragem pontual (*jitter*) manteve-se abaixo de 0.5%, provando que os mecanismos de exclusão mútua (`std::mutex`) no buffer compartilhado foram projetados corretamente, mitigando condições de corrida sem travar o processamento.

3.4 Execução Concorrente de Tarefas Periódicas e Assíncronas

A análise dos dados numéricos brutos gravados diretamente em disco permite avaliar a cronologia de preempção do kernel do Linux e o comportamento temporal síncrono do sistema.

```

[ODOMETRIA_20MS] Tempo de Execucao (Ci): 0.002 ms | Período Real (Ti): 20.181 ms
[ODOMETRIA_20MS] Tempo de Execucao (Ci): 0.001 ms | Período Real (Ti): 19.969 ms
[ODOMETRIA_20MS] Tempo de Execucao (Ci): 0.001 ms | Período Real (Ti): 20.040 ms
[COMANDO_NAV_80MS] Tempo de Execucao (Ci): 0.001 ms | Período Real (Ti): 80.127 ms
[ODOMETRIA_20MS] Tempo de Execucao (Ci): 0.000 ms | Período Real (Ti): 19.926 ms
[CONTROLE_PID_80MS] Tempo de Execucao (Ci): 0.001 ms | Período Real (Ti): 80.178 ms
[ODOMETRIA_20MS] Tempo de Execucao (Ci): 0.002 ms | Período Real (Ti): 20.058 ms
[LIDAR_RECONSTR_100MS] Tempo de Execucao (Ci): 0.001 ms | Período Real (Ti): 100.110 ms
[ODOMETRIA_20MS] Tempo de Execucao (Ci): 0.001 ms | Período Real (Ti): 20.006 ms
[ODOMETRIA_20MS] Tempo de Execucao (Ci): 0.001 ms | Período Real (Ti): 19.999 ms
[COMANDO_NAV_80MS] Tempo de Execucao (Ci): 0.001 ms | Período Real (Ti): 80.015 ms
[ODOMETRIA_20MS] Tempo de Execucao (Ci): 0.001 ms | Período Real (Ti): 19.952 ms
[CONTROLE_PID_80MS] Tempo de Execucao (Ci): 0.001 ms | Período Real (Ti): 79.949 ms
[ODOMETRIA_20MS] Tempo de Execucao (Ci): 0.001 ms | Período Real (Ti): 20.016 ms
[ODOMETRIA_20MS] Tempo de Execucao (Ci): 0.001 ms | Período Real (Ti): 20.003 ms
[LIDAR_RECONSTR_100MS] Tempo de Execucao (Ci): 0.001 ms | Período Real (Ti): 99.995 ms
[ODOMETRIA_20MS] Tempo de Execucao (Ci): 0.001 ms | Período Real (Ti): 19.996 ms
[COMANDO_NAV_80MS] Tempo de Execucao (Ci): 0.001 ms | Período Real (Ti): 79.982 ms
[CONTROLE_PID_80MS] Tempo de Execucao (Ci): 0.001 ms | Período Real (Ti): 79.993 ms
[ODOMETRIA_20MS] Tempo de Execucao (Ci): 0.000 ms | Período Real (Ti): 19.999 ms
[ODOMETRIA_20MS] Tempo de Execucao (Ci): 0.001 ms | Período Real (Ti): 20.006 ms
[ODOMETRIA_20MS] Tempo de Execucao (Ci): 0.001 ms | Período Real (Ti): 20.000 ms
[LIDAR_RECONSTR_100MS] Tempo de Execucao (Ci): 0.002 ms | Período Real (Ti): 100.011 ms
[ODOMETRIA_20MS] Tempo de Execucao (Ci): 0.000 ms | Período Real (Ti): 20.200 ms
[COMANDO_NAV_80MS] Tempo de Execucao (Ci): 0.002 ms | Período Real (Ti): 80.027 ms
[CONTROLE_PID_80MS] Tempo de Execucao (Ci): 0.001 ms | Período Real (Ti): 80.012 ms
[ODOMETRIA_20MS] Tempo de Execucao (Ci): 0.000 ms | Período Real (Ti): 19.788 ms
[COLETOR_BUFFER_EVENTO] Tempo de Execucao (Ci): 0.111 ms | Período Real (Ti): 323.688 ms
[ODOMETRIA_20MS] Tempo de Execucao (Ci): 0.001 ms | Período Real (Ti): 20.023 ms
[ODOMETRIA_20MS] Tempo de Execucao (Ci): 0.001 ms | Período Real (Ti): 20.000 ms
[ODOMETRIA_20MS] Tempo de Execucao (Ci): 0.001 ms | Período Real (Ti): 19.983 ms
[COMANDO_NAV_80MS] Tempo de Execucao (Ci): 0.002 ms | Período Real (Ti): 79.989 ms

```

Figura 4: Tempos de tarefas periodicas

Análise do Resultado: O arquivo de log bruto comprova a precisão do agendamento de tarefas cíclicas e o comportamento assíncrono do sistema. Observa-se a thread de mais alta frequência, `ODOMETRIA_20MS`, executando rigorosamente a cada 20 ms (19.926 ms, 20.058 ms, 20.006 ms), enquanto consome apenas 0.001 ms de tempo de CPU (C_i). De forma intercalada e harmônica, as tarefas `COMANDO_NAV_80MS` e `CONTROLE_PID_80MS` disparam em seus prazos exatos de 80 ms. A linha da thread tratadora de eventos `COLETOR_BUFFER_EVENTO` comprova o funcionamento assíncrono do padrão produtor-consumidor: ela permaneceu bloqueada em sua variável de condição sem gastar processamento por 323.688 ms (T_i) e, ao ser acordada pelo buffer, esvaziou a fila e executou a escrita persistente em disco em meros 0.111 ms (C_i).

3.5 Análise de Carga Real em Cenário Crítico de Inspeção Visual

O último teste validou o comportamento de concorrência do processador quando o robô encontra uma falha e dispara os módulos assíncronos mais pesados de escrita e computação gráfica.

```

[COLETOR_BUFFER_EVENTO] Tempo de Execucao (Ci): 0.186 ms
[COLETOR_BUFFER_EVENTO] Tempo de Execucao (Ci): 0.074 ms
[COLETOR_BUFFER_EVENTO] Tempo de Execucao (Ci): 0.112 ms
[COLETOR_BUFFER_EVENTO] Tempo de Execucao (Ci): 0.063 ms
[COLETOR_BUFFER_EVENTO] Tempo de Execucao (Ci): 0.090 ms
[COLETOR_BUFFER_EVENTO] Tempo de Execucao (Ci): 0.078 ms
[COLETOR_BUFFER_EVENTO] Tempo de Execucao (Ci): 0.086 ms
[COLETOR_BUFFER_EVENTO] Tempo de Execucao (Ci): 0.084 ms
[COLETOR_BUFFER_EVENTO] Tempo de Execucao (Ci): 0.059 ms
[COLETOR_BUFFER_EVENTO] Tempo de Execucao (Ci): 0.784 ms
[COLETOR_BUFFER_EVENTO] Tempo de Execucao (Ci): 0.084 ms
[CAMERA_IA_EVENTO] Tempo de Execucao (Ci): 28.065 ms | Per
[COLETOR_BUFFER_EVENTO] Tempo de Execucao (Ci): 0.075 ms
[COLETOR_BUFFER_EVENTO] Tempo de Execucao (Ci): 0.093 ms
[COLETOR_BUFFER_EVENTO] Tempo de Execucao (Ci): 0.110 ms
[COLETOR_BUFFER_EVENTO] Tempo de Execucao (Ci): 0.093 ms
[COLETOR_BUFFER_EVENTO] Tempo de Execucao (Ci): 0.067 ms
[COLETOR_BUFFER_EVENTO] Tempo de Execucao (Ci): 0.088 ms
[COLETOR_BUFFER_EVENTO] Tempo de Execucao (Ci): 0.083 ms
[COLETOR_BUFFER_EVENTO] Tempo de Execucao (Ci): 0.062 ms
[COLETOR_BUFFER_EVENTO] Tempo de Execucao (Ci): 0.080 ms
[COLETOR_BUFFER_EVENTO] Tempo de Execucao (Ci): 0.092 ms
[COLETOR_BUFFER_EVENTO] Tempo de Execucao (Ci): 0.101 ms
[COLETOR_BUFFER_EVENTO] Tempo de Execucao (Ci): 0.112 ms
[COLETOR_BUFFER_EVENTO] Tempo de Execucao (Ci): 0.097 ms
[COLETOR_BUFFER_EVENTO] Tempo de Execucao (Ci): 0.120 ms
[COLETOR_BUFFER_EVENTO] Tempo de Execucao (Ci): 0.094 ms
[CAMERA_IA_EVENTO] Tempo de Execucao (Ci): 25.350 ms | Per
[COLETOR_BUFFER_EVENTO] Tempo de Execucao (Ci): 0.076 ms
[COLETOR_BUFFER_EVENTO] Tempo de Execucao (Ci): 0.097 ms
[COLETOR_BUFFER_EVENTO] Tempo de Execucao (Ci): 0.081 ms
[COLETOR_BUFFER_EVENTO] Tempo de Execucao (Ci): 0.087 ms
[COLETOR_BUFFER_EVENTO] Tempo de Execucao (Ci): 0.188 ms
[COLETOR_BUFFER_EVENTO] Tempo de Execucao (Ci): 0.073 ms
[COLETOR_BUFFER_EVENTO] Tempo de Execucao (Ci): 0.171 ms
[COLETOR_BUFFER_EVENTO] Tempo de Execucao (Ci): 0.063 ms
[COLETOR_BUFFER_EVENTO] Tempo de Execucao (Ci): 0.067 ms
[COLETOR_BUFFER_EVENTO] Tempo de Execucao (Ci): 0.077 ms
[COLETOR_BUFFER_EVENTO] Tempo de Execucao (Ci): 0.092 ms
[COLETOR_BUFFER_EVENTO] Tempo de Execucao (Ci): 0.118 ms
[COLETOR_BUFFER_EVENTO] Tempo de Execucao (Ci): 0.067 ms
[COLETOR_BUFFER_EVENTO] Tempo de Execucao (Ci): 0.099 ms

```

Figura 5: Tempos de tarefas assincronas

Análise do Resultado: Este log documenta o exato instante em que o filtro de média móvel do

LIDAR detectou a anomalia estrutural e sinalizou a variável de condição da câmera. Conforme especificado pelo projeto, para simular um processamento pesado real de inteligência incorporada sem o uso de sleeps artificiais, a thread disparou seu loop matemático e reteve a CPU de forma legítima por 28.065 ms no primeiro evento e 25.350 ms no segundo.

A eficácia do sincronismo reside no fato de que, por termos liberado explicitamente o trinco de exclusão mútua (`lock.unlock()`) antes de chamar a função de IA, a thread do Coletor de Dados pôde continuar a ser preemptada e executada concorrentemente no meio do processo (registrando tempos de gravação síncronos entre 0.059 ms e 0.120 ms). Isso demonstra de forma cabal que o sistema está livre de inversão de prioridades ou travamentos indesejados. # Análise de Capacidade e Escalabilidade do Sistema

Esta seção apresenta a validação analítica da capacidade de processamento do sistema embarcado, fundamentada nos conceitos de escalonabilidade em sistemas de tempo real rígido e nos dados empíricos de pior caso extraídos das medições de telemetria.

3.6 Levantamento dos Piores Tempos de Execução (C_i)

Com base nos logs reais coletados a partir da execução concorrente do sistema embarcado, foram mapeados os piores tempos de computação (*Worst-Case Execution Time* - WCET) para cada tarefa do sistema:

- **Tarefas Cíclicas Base:**

- $C_{odo} = 0.027$ ms (Cálculo de distância / Odometria)
- $C_{com} = 0.025$ ms (Comando de Navegação)
- $C_{pid} = 0.013$ ms (Controle de Navegação PID)
- $C_{recon} = 0.029$ ms (Reconstrução da superfície do teto / LIDAR)

- **Tarefas por Evento (Assíncronas):**

- $C_{coletor} = 0.147$ ms (Esvaziamento de buffer e I/O de escrita no arquivo CSV)
- $C_{camera} = 25.719$ ms (Loop numérico contínuo simulando processamento pesado de IA)

3.7 Modelo Teórico de Escalonabilidade Avançado

Para a modelagem de tarefas esporádicas e assíncronas (disparadas estritamente por eventos), a extensão da teoria clássica de tempo real de Liu e Layland exige a definição do intervalo mínimo entre eventos (T_{evento}). Este intervalo baliza o pior cenário de estresse físico que o robô de inspeção pode enfrentar ao navegar pelo túnel:

- **Coletor de Dados:** No cenário de estresse máximo, assume-se que o buffer compartilhado recebe dados de telemetria continuamente, de modo que o Coletor é sinalizado e acordado a cada ciclo de 20 ms ($T_{coletor} = 20$ ms).
- **Inspeção por Câmera:** O pior cenário físico possível ocorre caso o teto do túnel apresente severo dano estrutural sequencial, fazendo com que o filtro do LIDAR detecte uma nova falha imediatamente após a thread da câmera terminar o processamento da inspeção anterior. Dado que a execução pesada da IA consome cerca de 25.7 ms, assume-se um disparo crítico a cada 30 ms ($T_{camera} = 30$ ms).

3.8 Cálculo do Período Mínimo das Tarefas Cíclicas ($T_{\min}$)

Assumindo um cenário de dimensionamento limite do processador onde todas as tarefas cíclicas base passam a rodar sob estresse na mesma frequência máxima com um período limite comum $T_{\min}$, a inequação da taxa de utilização global da CPU (U) sob a concorrência assíncrona da Câmera e do Coletor é modelada por:

$$U = \frac{C_{\text{odo}} + C_{\text{com}} + C_{\text{pid}} + C_{\text{recon}}}{T_{\min}} + \frac{C_{\text{coletor}}}{T_{\text{coletor}}} + \frac{C_{\text{camera}}}{T_{\text{camera}}} \leq 1.0$$

Substituindo os valores experimentais mapeados nos logs e as restrições físicas de pior caso adotadas:

$$\frac{0.027 + 0.025 + 0.013 + 0.029}{T_{\min}} + \frac{0.147}{20} + \frac{25.719}{30} \leq 1.0$$

$$\frac{0.094 \text{ ms}}{T_{\min}} + 0.00735 + 0.8573 \leq 1.0$$

A análise matemática isolada revela o impacto do algoritmo de inteligência incorporada da câmera, que abocanha sozinho 85.73% da capacidade de processamento da CPU no instante em que está ativo. Agrupando a carga persistente fixada pelas tarefas por evento ($0.00735 + 0.8573 = 0.86465$), isola-se o limite de amostragem síncrono:

$$\frac{0.094 \text{ ms}}{T_{\min}} + 0.86465 \leq 1.0$$

$$\frac{0.094 \text{ ms}}{T_{\min}} \leq 1.0 - 0.86465$$

$$\frac{0.094 \text{ ms}}{T_{\min}} \leq 0.13535$$

$$T_{\min} \geq \frac{0.094}{0.13535} \Rightarrow T_{\min} \geq \mathbf{0.694 \text{ ms}}$$

3.9 Discussão Teórica e Restrições de Hardware

O resultado analítico demonstra que, matematicamente, o menor tempo limite que as tarefas cíclicas de controle e navegação poderiam adotar sem causar o colapso e a perda de prazos (*deadlines*) do sistema é de **0.694 ms**.

Contudo, a engenharia de sistemas embarcados impõe restrições práticas de hardware que tornam este limite puramente numérico inviável em cenários reais: 1. **Overhead de Troca de Contexto (*Context Switch*)**: Em períodos inferiores a 1 ms, o tempo gasto pelo escalonador do sistema operacional para salvar registradores e alternar o contexto das threads passa a ser comparável ao tempo de execução útil das tarefas, gerando sobrecarga (*thrashing*). 2. **Resolução do Timer**

do Kernel Linux: O sistema operacional convencional gerencia o tempo através de interrupções cíclicas de hardware (*ticks*). Isto limita o agendamento determinístico estável de threads a uma resolução mínima real de 1 **ms**. Tentar operar abaixo desse limite resultaria em um *jitter* temporal destrutivo.

Desta forma, conclui-se que os períodos adotados no projeto (20 ms, 80 ms e 100 ms) constituem uma decisão de engenharia precisa e segura. Eles oferecem uma margem de proteção robusta que garante o determinismo temporal síncrono e cede a prioridade da CPU de forma imediata à Câmera de IA sempre que uma anomalia estrutural for detectada.

4 Conclusão e Discussões

5 Apêndice